Universidad Autónoma de Madrid

Escuela Politécnica Superior

25|26

# Trabajo AprendizajeAutomático III

## Machine learning project: Generative AI system



Jimena de Prado González

Escuela Politécnica Superior
Universidad Autónoma de Madrid
C\Francisco Tomás y Valiente nº 11

excelencia Campus Internacional UAM CSIC

www.uam.es

**UNIVERSIDAD AUTÓNOMA DE MADRID**
**ESCUELA POLITÉCNICA SUPERIOR**







**Bachelor as Grado en Ciencia e Ingeniería de Datos**

# Trabajo AprendizajeAutomático III

## Machine learning project: Generative AI system

**Author: Jimena de Prado González**

**mayo 2026**

**Some rights reserved**

This work is licensed under a Creative Commons
Attribution-NonCommercial-ShareAlike 3.0 License.
http://creativecommons.org/licenses/by-nc-sa/3.0/
You are free to share (copy, distribute and transmit) and to modify the work
under the following conditions:

- You must attribute the work in the manner specified by the author or licensor (but not in any way that suggests that they endorse you or your use of the work).
- You may not use this work for commercial purposes.
- If you alter, transform, or build upon this work, you may distribute the resulting work only under the same or similar license to this one.

RIGHTS RESERVED
© 3 de mayo de 2026 por UNIVERSIDAD AUTÓNOMA DE MADRID
Francisco Tomás y Valiente, nº 1
Madrid, 28049
Spain

**Jimena de Prado González**
**Machine learning project: Generative AI system**

**Jimena de Prado González**
C\ Francisco Tomás y Valiente Nº 11

PRINTED IN SPAIN

# Abstract

This project develops a generative AI system for image generation using score-based diffusion models. We focus on the MNIST dataset, which contains grayscale images of handwritten digits, and train neural networks to learn the score of noisy image distributions. Instead of modelling the image probability density directly, our approach estimates the direction in which noisy samples should move to become more likely under the data distribution.

The system implements different forward diffusion processes and sampling methods. In particular, we compare Variance Exploding diffusion with an exponential noise schedule, Variance Preserving diffusion with a linear and cosine schedules. For generation, we use Euler-Maruyama sampling, Predictor-Corrector sampling, and the Probability Flow ODE. The trained models are saved as checkpoint files and later loaded in the main notebook for sampling, evaluation and controllable generation experiments.

The results show that the choice of noise schedule has a clear impact on generation quality. The VP-linear configuration performs worse because the linear schedule tends to destroy useful image information less effectively for our setting, while VE and VP-cosine produce more coherent digit-like samples. Predictor-Corrector sampling generally improves visual quality, although it is slower and sensitive to the corrector step size. The Probability Flow ODE is useful for likelihood estimation, but in our experiments it performs worse as a visual sampler.

We evaluate the models using Bits per Dimension, Fréchet Inception Distance and Inception Score. Since FID and IS rely on an Inception network trained on natural RGB images, we adapt MNIST images to three channels and interpret these metrics carefully. We also implement class-conditional sampling by adding the digit label to the ScoreNet, and image imputation by applying a binary mask and modifying the reverse samplers to preserve the observed pixels. Overall, the project shows that score-based diffusion models can generate and control MNIST-like images, while also highlighting the importance of schedules, samplers and evaluation choices.

# Table of Contents


**1 Introduction** **1**

1.1 Context and Motivation . . . . . 1

1.2 Objectives and Developed Work . . . . . 1

1.3 Report Structure . . . . . 1

**2 State of the Art and Theoretical Background** **3**

2.1 Problem Definition . . . . . 3

2.2 Generative Modelling for Image Synthesis . . . . . 3

2.3 Score-Based Diffusion Models . . . . . 4

2.4 Score Network Architecture . . . . . 5

2.5 Forward Diffusion Processes . . . . . 5

2.6 Noise Schedules . . . . . 7

2.7 Reverse-Time Generation . . . . . 8

2.8 Quality Metrics . . . . . 10

2.9 Controllable Generation . . . . . 12

**3 System Design and Implementation** **14**

3.1 Notebook Organisation . . . . . 14

3.2 Implementation of the Diffusion Models . . . . . 14

3.3 Sampling Implementation . . . . . 15

3.4 Quality Metrics Implementation . . . . . 15

3.5 Controllable Generation . . . . . 17

**4 Results** **20**

4.1 Unconditional Generation Results . . . . . 20

4.2 Quantitative Evaluation . . . . . 22

4.3 Controllable Generation Results . . . . . 24

**5 Conclusions** **28**

**Bibliography** **30**

# Lists

## List of codes


3.1 Hutchinson estimator for BPD . . . . . 16
3.2 Preprocessing for Inception-based metrics . . . . . 17
3.3 Class embedding in the conditional ScoreNet . . . . . 18
3.4 Conditioning the ScoreNet on the class label . . . . . 18
3.5 Euler-Maruyama imputation update . . . . . 19
3.6 Probability Flow ODE imputation update . . . . . 19


## List of figures


4.1 VE Euler-Maruyama . . . . . 20
4.2 VE Predictor-Corrector . . . . . 20
4.3 VE Probability Flow ODE . . . . . 21
4.4 VP-linear Predictor-Corrector . . . . . 21
4.5 VP-cosine Euler-Maruyama . . . . . 22
4.6 VP-cosine Predictor-Corrector . . . . . 22
4.7 VP-cosine Probability Flow ODE . . . . . 22
4.8 Conditional sampling for digit 3 . . . . . 25
4.9 Conditional sampling for digit 7 . . . . . 25
4.10 Conditional sampling for digit 4 . . . . . 25
4.11 Masked images for imputation . . . . . 26
4.12 VE Euler-Maruyama imputation . . . . . 26
4.13 VP-linear Predictor-Corrector imputation . . . . . 27
4.14 VP-cosine ODE imputation . . . . . 27


## List of tables


4.1 BPD results . . . . . 23
4.2 FID results . . . . . 23
4.3 Inception Score results . . . . . 24

# 1

# Introduction

## 1.1. Context and Motivation

Generative AI focuses on models that can create new data samples with similar characteristics to the examples used during training. In this project, we develop a generative AI system for image generation using score-based diffusion models. The system is applied to the MNIST dataset, which contains grayscale images of handwritten digits.

The main idea behind the project is to train a neural network to estimate the score of noisy image distributions. This score is then used during the reverse diffusion process to transform random noise into digit-like images. Although MNIST is a simple dataset, it allows us to study the main components of diffusion-based generation in a controlled setting.

## 1.2. Objectives and Developed Work

The objective of this project is to understand how score-based diffusion models can be used to generate new images. More specifically, we aim to learn how to train a neural network to estimate the score of noisy image distributions, and how stochastic differential equations can describe the process of transforming real images into noise and then reversing this process to generate new samples.

To achieve this, we implemented and compared different forward diffusion processes: VE diffusion with an exponential schedule, VP diffusion with a linear schedule, and VP diffusion with a cosine schedule. For image generation, we used Euler-Maruyama, Predictor-Corrector and Probability Flow ODE sampling. We also evaluated the generated samples using BPD, FID and Inception Score.

Finally, we extended the system with two controllable generation tasks: class-conditional sampling and image imputation. In the first case, we modified the model to include the target digit label. In the second case, we adapted the reverse samplers to complete masked MNIST images while preserving the observed pixels.

## 1.3. Report Structure

The report is organised as follows. Chapter 2 presents the theoretical background of score-based diffusion models. Chapter 3 explains the system design and implementation. Chapter 4 discusses the

experimental results and evaluation. Chapter 5 summarises the main conclusions, limitations and possible future improvements.

# 2

# State of the Art and Theoretical Background

## 2.1. Problem Definition

The objective of this project is to build a generative AI system for image generation using the MNIST dataset. This dataset contains images of handwritten digits, which we use as examples of the target image distribution that our model aims to learn. In other words, we train a model to capture the structure of the probability distribution of the images in the MNIST dataset, and then generate new images that resemble these handwritten digits. The system is based on score-based diffusion models, where images are progressively corrupted by noise through a forward stochastic process and later generated by simulating the corresponding reverse-time process.

## 2.2. Generative Modelling for Image Synthesis

Generative modelling for image synthesis aims to learn how to create new images that follow the same visual patterns as a given dataset. The main challenge with images is that their distribution is difficult to model explicitly, since each one is represented by many pixel values, and these pixels are strongly dependent on one another.

Traditional generative modelling approaches usually represent probability distributions in one of two ways. Likelihood-based models try to explicitly model the probability density of the data, but they often require architectural restrictions or approximations in order to make likelihood computation tractable. Implicit generative models, such as GANs, avoid explicit likelihood estimation and instead learn a sampling mechanism, but they usually rely on adversarial training, which can be unstable and may suffer from mode collapse. Score-based generative modelling provides an alternative approach: instead of learning the probability density itself, we learn the direction in which an image should be modified to become more likely under the data distribution [1]. To find this direction, we train a neural network to approximate the score function of noisy image distributions.

## 2.3. Score-Based Diffusion Models

Score-based diffusion models represent a probability distribution through its score function, defined as the gradient of the log-density with respect to the input image [1]:

$$\nabla_{\mathbf{x}} \log p_t(\mathbf{x}).$$

This indicates the direction in image space where the probability density increases the most. Instead of estimating the full density $p_t(\mathbf{x})$, which is difficult in high-dimensional image spaces, we train a neural network $\mathbf{s}_\theta(\mathbf{x}, t)$ to approximate this score function:

$$\mathbf{s}_\theta(\mathbf{x}, t) \approx \nabla_{\mathbf{x}} \log p_t(\mathbf{x}).$$

As a result, the model does not need to compute the exact probability of each image during training. Instead, it only needs to learn a vector field that points noisy images towards regions of higher probability. In our implementation, the model is trained using denoising score matching, following the general idea of learning scores from noise-perturbed data [**?**]. For each MNIST image $\mathbf{x}_0$, we sample a random time $t$ and add Gaussian noise according to the forward diffusion process:

$$\mathbf{x}(t) = \mu_t(\mathbf{x}_0) + \sigma_t \mathbf{z}, \qquad \mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}).$$

Since the noisy image is obtained by adding Gaussian noise to the original image, the conditional distribution of $\mathbf{x}(t)$ given $\mathbf{x}_0$ is Gaussian. This allows us to compute the target score explicitly. For a Gaussian distribution, the score points from the noisy sample back toward the mean of the distribution. Therefore, the target score is

$$\nabla_{\mathbf{x}(t)} \log p_{0t}(\mathbf{x}(t) \mid \mathbf{x}_0) = -\frac{\mathbf{x}(t) - \mu_t(\mathbf{x}_0)}{\sigma_t^2} = -\frac{\mathbf{z}}{\sigma_t}.$$

This gives us a direct training target for the ScoreNet without needing to know the true data distribution. Therefore, we train the net using pairs of noisy images and their corresponding noise values. The training objective we used in our model is:

$$\mathcal{L}(\theta) = \mathbb{E}_{t,\mathbf{x}_0,\mathbf{z}} \left[ \|\sigma_t \mathbf{s}_\theta(\mathbf{x}(t), t) + \mathbf{z}\|_2^2 \right].$$

Which lets the network predict the direction that removes noise from the corrupted image. Once the ScoreNet has learned this time-dependent score, we can use it during the reverse diffusion process to progressively transform random noise into images that resemble handwritten digits.

## 2.4. Score Network Architecture

As we already mentioned, the ScoreNet is the neural network used to approximate the score function. Its input is a noisy MNIST image $\mathbf{x}(t)$ and the corresponding time value $t$. Its output is an estimate of the score:

$$\mathbf{s}_\theta(\mathbf{x}(t), t) \approx \nabla_{\mathbf{x}} \log p_t(\mathbf{x}(t)).$$

This means that, for each noise level, the network learns the direction in which the noisy image should be modified to become more likely under the image distribution. Since the score changes with the amount of noise, the network must be conditioned on time. To do this, the scalar time value $t$ is transformed into a higher-dimensional embedding using Gaussian Random Fourier Features:

$$\gamma(t) = [\sin(2\pi t\mathbf{W}), \cos(2\pi t\mathbf{W})]\,,$$

where $\mathbf{W}$ is a vector of fixed random frequencies. This gives the network a richer representation of the noise level than using $t$ directly as a single number. The resulting time embedding is then passed through a dense layer and injected into the convolutional blocks of the network.

The image itself is processed with a U-Net architecture. The encoder progressively reduces the spatial resolution and extracts image features, while the decoder reconstructs an output with the same size as the input image. This type of convolutional architecture is useful for image data because it can exploit local spatial structure, such as strokes, edges and digit shapes [2]. Skip connections link the encoder and decoder layers, allowing the model to preserve this local information during the reconstruction of the score estimate.

Finally, the network output is divided by the marginal standard deviation $\sigma_t$. This normalization makes the predicted score consistent with the scale of the noise at each time step.

## 2.5. Forward Diffusion Processes

The forward diffusion process defines how noise is progressively added to an original image. Starting from a clean image $\mathbf{x}_0$, the process generates a noisy version $\mathbf{x}(t)$ for each time value $t \in [0, T]$. As $t$ increases, more noise is added, and the image gradually loses its visual structure. In continuous time, the forward process can be described using a stochastic differential equation (SDE) [3]:

$$d\mathbf{x}(t) = \mathbf{f}(\mathbf{x}(t), t)dt + g(t)d\mathbf{W}(t),$$

where $\mathbf{f}(\mathbf{x}(t), t)$ is the drift term, $g(t)$ is the diffusion coefficient, and $\mathbf{W}(t)$ is a Wiener process. The

drift controls the deterministic change of the image, while the diffusion coefficient controls the amount of random noise added at each time step.

In our project, we consider two types of diffusion processes: Variance Exploding (VE) diffusion and Variance Preserving (VP) diffusion. Both processes transform clean images into noise, but they do so in different ways.

### 2.5.1. Variance Exploding Diffusion

The Variance Exploding (VE) diffusion process is based on adding Gaussian noise with increasing variance. In this case, the drift term is zero, so the forward SDE is

$$d\mathbf{x}(t) = g(t)d\mathbf{W}(t).$$

This means that the image does not move deterministically away from its original value; instead, it is corrupted only by the accumulation of random noise. The diffusion coefficient is defined as $g(t) = \sigma^t$ where $\sigma$ controls how quickly the noise level increases. Since the drift is zero, the conditional mean of the noisy image remains equal to the original image: $\mu_t(\mathbf{x}_0) = \mathbf{x}_0$. Given the original image $\mathbf{x}_0$, the noisy image follows a Gaussian distribution, $p_{0t}(\mathbf{x}(t) \mid \mathbf{x}_0) = \mathcal{N}(\mathbf{x}_0, \sigma_t^2\mathbf{I})$, where the marginal standard deviation is:

$$\sigma_t = \sqrt{\frac{\sigma^{2t} - 1}{2\log\sigma}}.$$

Therefore, a noisy image can be sampled as

$$\mathbf{x}(t) = \mathbf{x}_0 + \sigma_t\mathbf{z}, \qquad \mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}).$$

The main advantage of the VE process is that it is simple and easy to interpret: the original image is progressively corrupted by adding stronger Gaussian noise. However, because the variance increases over time, the process can become numerically unstable when the noise level is too large, especially near the final time.

### 2.5.2. Variance Preserving Diffusion

The Variance Preserving (VP) diffusion process follows a different strategy. Instead of only adding noise, it also progressively reduces the strength of the original image signal. Its forward SDE is

$$d\mathbf{x}(t) = -\frac{1}{2}\beta(t)\mathbf{x}(t)dt + \sqrt{\beta(t)}d\mathbf{W}(t),$$

where $\beta(t)$ is a time-dependent noise schedule. The drift term $-\frac{1}{2}\beta(t)\mathbf{x}(t)$, gradually pulls the image values toward zero, while the diffusion term $\sqrt{\beta(t)}d\mathbf{W}(t)$adds Gaussian noise. The conditional distribution of the noisy image is also Gaussian. For the VP process, the conditional mean is

$$\mu_t(\mathbf{x}_0) = \exp\left(-\frac{1}{2}\int_0^t \beta(s)ds\right)\mathbf{x}_0,$$

and the marginal standard deviation is

$$\sigma_t = \sqrt{1 - \exp\left(-\int_0^t \beta(s)ds\right)}.$$

Thus, a noisy image can be written as

$$\mathbf{x}(t) = \mu_t(\mathbf{x}_0) + \sigma_t\mathbf{z}, \qquad \mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}).$$

The main advantage of the VP process is that the variance remains controlled during diffusion, which makes it closely related to classical denoising diffusion models. However, its performance depends strongly on the choice of the noise schedule $\beta(t)$.

## 2.6. Noise Schedules

The noise schedule determines how the amount of noise changes over time during the forward diffusion process. This choice is important because it controls how quickly the original image information is destroyed. If noise is added too fast, the model may lose useful structure early in the process. If noise is added too slowly, the final distribution may not be close enough to simple Gaussian noise.

For the Variance Exploding (VE) process, we have used an exponential schedule for the diffusion coefficient, $g(t) = \sigma^t$. For the Variance Preserving (VP) process, we used schedules defined through the noise rate $\beta(t)$, considering both linear and cosine variants.

### 2.6.1. Exponential Schedule

As we defined previously, in the VE formulation the image mean remains fixed and the corruption comes only from the growth of the noise variance. For this reason, the schedule must directly control how the variance increases over time. We used an exponential diffusion coefficient, $g(t) = \sigma^t$, because it produces a slow increase in noise at the beginning and a much stronger increase near the final time, which is ideal for VE. As a result, the model observes images at many different noise scales: almost clean images for small values of $t$, and almost pure noise for values of $t$ close to $T$.

The accumulated noise level is

$$\sigma_t = \sqrt{\frac{\sigma^{2t} - 1}{2 \log \sigma}}.$$

Therefore, the noisy image can be written as $\mathbf{x}(t) = \mathbf{x}_0 + \sigma_t \mathbf{z}$. This shows the main idea of VE diffusion: the original image stays centred around $\mathbf{x}_0$, but as the variance increases progressively, the image structure ends up completely hidden by noise.

### 2.6.2. Linear Schedule

This schedule is defined through the noise rate $\beta(t)$. This coefficient controls both the deterministic decay of the image signal and the amount of Gaussian noise added at each time step. The linear schedule is defined as $\beta(t) = \beta_{\text{mín}} + t(\beta_{\text{máx}} - \beta_{\text{mín}})$, where $\beta_{\text{mín}}$ and $\beta_{\text{máx}}$ control the initial and final noise rates.

This schedule is simple and easy to interpret. At the beginning of the process, the image is only slightly corrupted, while at later times the noise becomes stronger. However, the visual degradation of an image is not necessarily linear, so the schedule may destroy relevant image information too quickly or too slowly depending on the chosen parameters.

### 2.6.3. Cosine Schedule

It defines how much of the original signal is preserved over time through the cumulative term $\bar{\alpha}(t)$:

$$\bar{\alpha}(t) = \frac{\cos^2\left(\frac{t+s}{1+s}\frac{\pi}{2}\right)}{\cos^2\left(\frac{s}{1+s}\frac{\pi}{2}\right)},$$

where $s$ is a small offset used for numerical stability. The corresponding standard deviation is $\sigma_t = \sqrt{1 - \bar{\alpha}(t)}$. Compared with the linear schedule, the cosine schedule tends to preserve more image information during the early stages of diffusion and adds stronger noise later. This can make the denoising task smoother for the model. However, it requires more numerical care near the final time, where the noise level changes very rapidly.

## 2.7. Reverse-Time Generation

Once the ScoreNet has been trained, we can use it to generate new images. The forward diffusion process transforms clean MNIST images into noise, while the reverse-time process tries to undo this

transformation. Therefore, generation starts from a random noise sample and progressively denoises it until it becomes a realistic handwritten digit.

In continuous time, the reverse process is also described by an SDE, using the reverse-time formulation of diffusion processes. If the forward process is

$$d\mathbf{x}(t) = \mathbf{f}(\mathbf{x}(t), t)dt + g(t)d\mathbf{W}(t),$$

then the reverse-time SDE uses the score function to move samples toward regions of higher probability:

$$d\mathbf{x}(t) = \left[\mathbf{f}(\mathbf{x}(t), t) - g(t)^2 \nabla_{\mathbf{x}} \log p_t(\mathbf{x}(t))\right] dt + g(t)d\bar{\mathbf{W}}(t).$$

This way, the learned ScoreNet 'guides' the denoising trajectory from pure noise back to the image distribution.

### 2.7.1. Euler-Maruyama Sampling

Euler-Maruyama is the simplest numerical method used to simulate the reverse SDE. Starting from an initial noisy sample at $t = T$, the method moves backwards in small time steps until it reaches $t \approx 0$. At each step, the image is updated using the reverse drift given by the score model and an additional random noise term. The update can be written as

$$\mathbf{x}_{n+1} = \mathbf{x}_n + \left[\mathbf{f}(\mathbf{x}_n, t_n) - g(t_n)^2 \mathbf{s}_\theta(\mathbf{x}_n, t_n)\right] \Delta t + g(t_n)\sqrt{|\Delta t|}\mathbf{z}_n,$$

where $\mathbf{z}_n \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$. This method is easy to implement and computationally efficient, but its quality depends strongly on the number of time steps. If the step size is too large, the generated images may not be clear digit shapes.

### 2.7.2. Predictor-Corrector Sampling

Predictor-Corrector sampling improves the reverse generation process by combining two types of updates. The predictor step follows the reverse SDE, usually using Euler-Maruyama. The corrector step then refines the sample using Langevin dynamics, which moves the image further in the direction indicated by the score. A simple corrector update is

$$\mathbf{x} \leftarrow \mathbf{x} + \epsilon \mathbf{s}_\theta(\mathbf{x}, t) + \sqrt{2\epsilon}\mathbf{z}, \qquad \mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}).$$

The predictor gives the general denoising trajectory, while the corrector improves the sample at

each noise level. This usually leads to better image quality, because the model has more opportunities to adjust the sample toward high-probability regions. However, it is also slower than Euler-Maruyama, since each corrector step requires additional evaluations of the ScoreNet.

### 2.7.3. Probability Flow ODE

The Probability Flow ODE provides a deterministic alternative to the reverse SDE. Instead of adding random noise during sampling, it follows an ordinary differential equation with the same marginal distributions as the stochastic process. Its dynamics are given by

$$d\mathbf{x}(t) = \left[\mathbf{f}(\mathbf{x}(t), t) - \frac{1}{2} g(t)^2 \mathbf{s}_\theta(\mathbf{x}(t), t)\right] dt.$$

The main difference from the reverse SDE is the absence of the stochastic term $g(t)d\bar{\mathbf{W}}(t)$. As a result, if we start from the same initial noise, the Probability Flow ODE always produces the same generated image. This method is important because it can be used not only for generation, but also for likelihood estimation, which is why the Probability Flow ODE is used later to compute bits per dimension.

## 2.8. Quality Metrics

To evaluate the performance of the generative model, visual inspection might not be enough. That's why we also consider three metrics: Bits per Dimension (BPD), Fréchet Inception Distance (FID) and Inception Score (IS). Each metric captures a different aspect of model quality.

### 2.8.1. Bits per Dimension

Bits per Dimension (BPD) is a likelihood-based metric. It measures how well the model assigns probability to real images, normalized by the number of image dimensions. For an image $\mathbf{x}$ with $D$ dimensions, BPD is defined as

$$\text{BPD} = -\frac{\log p_\theta(\mathbf{x})}{D \log 2}.$$

A lower BPD means that the model assigns higher likelihood to the data, so lower values are better. As we commented earlier, BPD is computed using the Probability Flow ODE. This is because the ODE defines a deterministic transformation between the data distribution and the noise distribution. Therefore, we can follow this deterministic path from the image to the noise distribution and track how its probability changes along the trajectory. In practice, this requires accumulating the divergence of the ODE drift along the trajectory, which makes BPD more computationally expensive than purely visual

metrics. We should keep in mind that good likelihood does not always imply visually better samples. For this reason, BPD should be interpreted together with other metrics and with visual inspection.

### 2.8.2. Fréchet Inception Distance

With Fréchet Inception Distance (FID), both sets of images are first represented using features extracted from a pretrained Inception network, usually Inception-v3, which is based on deep convolutional image representations [2]. These feature distributions are then approximated as Gaussian distributions. If the real images have feature mean and covariance $(\mu_r, \Sigma_r)$, and the generated images have $(\mu_g, \Sigma_g)$, the FID is defined as

$$\mathrm{FID} = \|\mu_r - \mu_g\|_2^2 + \mathrm{Tr}\left(\Sigma_r + \Sigma_g - 2(\Sigma_r\Sigma_g)^{1/2}\right).$$

A lower FID means that the generated images are closer to the real images in feature space, so lower values are better. FID is useful because it evaluates the quality of a whole set of generated images rather than only individual samples. However, it was originally designed for natural RGB images, not for the simple grayscale digits that we used in this project. Therefore, MNIST images must be adapted to the expected input format of the Inception network, and the resulting values should be interpreted carefully.

### 2.8.3. Inception Score

The Inception Score (IS) also uses Inception-v3. This network was originally trained on ImageNet, a large dataset of natural RGB images. The metric uses the predicted label distribution $p(y \mid \mathbf{x})$ produced by the Inception network. A good generated image should give a confident prediction, meaning that $p(y \mid \mathbf{x})$ is concentrated on one class. At the same time, the full set of generated images should be diverse, so the marginal distribution $p(y)$ should contain different classes. Formally, it is defined as

$$\mathrm{IS} = \exp\left(\mathbb{E}_{\mathbf{x}}\left[D_{KL}\left(p(y \mid \mathbf{x}) \| p(y)\right)\right]\right).$$

A higher Inception Score means that the generated images are both easier to classify and more diverse. However, as with FID, this metric was originally designed for natural RGB images, not for simple grayscale datasets such as MNIST. So, the results should also be carefully interpreted.

## 2.9. Controllable Generation

Diffusion models can also be adapted to guide the generation process toward a desired output. In this project, we consider two forms of controllable generation: class-conditional sampling and image imputation. In both cases, we do not only generate images from random noise, but also impose additional information that restricts or guides the reverse diffusion process.

### 2.9.1. Class-Conditional Sampling

In class-conditional sampling, we generate images that belong to a specific digit class. Instead of learning only the unconditional score $\nabla_{\mathbf{x}} \log p_t(\mathbf{x})$, the model learns a conditional score that also depends on the class label $y$:

$$\mathbf{s}_\theta(\mathbf{x}(t), t, y) \approx \nabla_{\mathbf{x}} \log p_t(\mathbf{x}(t) \mid y).$$

This means that, during reverse-time generation, the ScoreNet does not only denoise the image, but also guides it toward the selected digit class. For example, if we condition the model on the label $y = 3$, the generated sample should progressively evolve from random noise into an image that resembles a handwritten three.

This approach is useful because we can choose which type of digit we want to generate. Its main limitation is that the model must learn the relationship between noisy images, time values and labels, which makes the learning task more complex than unconditional generation.

### 2.9.2. Image Imputation

Image imputation consists of reconstructing missing parts of an image while keeping the observed parts fixed. We start from an incomplete MNIST image, and use the diffusion model to fill in the unknown region in a probable way.

We define a binary mask $\mathbf{m}$, where observed pixels are marked with value $1$ and missing pixels with value $0$. The goal is to generate the missing region while preserving the known pixels. During reverse-time generation, the model denoises the whole image, but after each step we enforce consistency with the observed part. A simple way to express this is

$$\mathbf{x}(t) = \mathbf{m} \odot \mathbf{x}_{obs}(t) + (1 - \mathbf{m}) \odot \mathbf{x}_{gen}(t),$$

where $\odot$ denotes element-wise multiplication, $\mathbf{x}_{obs}(t)$ is the noisy version of the observed pixels, and $\mathbf{x}_{gen}(t)$ is the image produced by the reverse process.

The quality of the result depends on how much information is available in the observed region and on whether the missing part can be inferred from the remaining digit structure.

# 3

# System Design and Implementation

## 3.1. Notebook Organisation

The implementation of this project is contained in the notebook named 'project', which brings together the trained models, sampling methods, quality metrics and controllable generation experiments. The notebook is organised in the same order as the experimental workflow. First, we load the required libraries, utility functions and trained ScoreNet models. Then, we evaluate the three diffusion configurations considered in this project: VE diffusion with an exponential schedule, VP diffusion with a linear and cosine schedule.

For each diffusion configuration, we generate images using the three reverse-time methods discussed in the previous chapter: Euler-Maruyama, Predictor-Corrector and Probability Flow ODE. After this, we compute the quality metrics we already described: BPD, FID and Inception Score. Finally, the notebook includes the controllable generation experiments: class-conditional sampling and image imputation.

## 3.2. Implementation of the Diffusion Models

The diffusion models were trained in separate notebooks, one for each configuration considered in the project. In particular, we used one notebook for the VE model with an exponential schedule, one notebook for the VP model with a linear schedule, and lastly one for the VP model with a cosine schedule.

Each training notebook follows the same general pipeline. First, we import the required libraries and define the device used for training, using GPU acceleration when available. Then, we load the MNIST dataset and create the data loader that provides batches of handwritten digit images. After this, we define the corresponding diffusion process by specifying its drift coefficient, diffusion coefficient, marginal mean $\mu_t$, and marginal standard deviation $\sigma_t$.

Once the diffusion process is defined, we initialise the ScoreNet model. The same general architecture is used across the experiments, but the time-dependent noise scale changes depending on the diffusion process. During training, each batch of MNIST images is randomly perturbed at different time values. The ScoreNet receives noisy MNIST images and is trained by approximating the score matching loss function. This does not mean that the network directly reconstructs clean images; rather,

the loss guides it to learn the score needed for the reverse diffusion process. In the training loop, we compute the loss for each batch, perform backpropagation, and update the model parameters using the optimizer.

After training, the learned weights of the ScoreNet are saved in separate files, and allow us to reuse them for the denoising process. The saved models and training notebooks are stored in a separate directory, named `trained_models/`, and are later loaded in the main project notebook for sampling.

## 3.3. Sampling Implementation

The saved models are loaded in the main 'project' notebook and used for generation. For each trained configuration, we apply the three reverse-time methods considered in the project: Euler-Maruyama sampling, Probability Flow ODE sampling, and Predictor-Corrector sampling.

The sampling process starts from Gaussian noise and uses the trained ScoreNet to guide the reverse trajectory toward the MNIST image distribution. In the notebook, we use `n_images = 100` from the `data_loader` for the sampling and evaluation workflow. However, for visual clarity, only the first five generated images are plotted in the report. This allows us to show representative results without making the figures too large. The outputs are visualised as image grids, and will be discussed further in this report.

## 3.4. Quality Metrics Implementation

To evaluate the generated samples, we implemented three quality metrics: Bits per Dimension (BPD), Fréchet Inception Distance (FID), and Inception Score (IS). These metrics were computed in the main project notebook after generating the corresponding samples.

### 3.4.1. Bits per Dimension

For BPD, we created our own implementation because it is directly related to the Probability Flow ODE used in score-based diffusion models. We use real images from the data loader and follow their deterministic trajectory through the Probability Flow ODE. Along this trajectory, we accumulate the change in log-probability. Since computing the exact divergence of the ODE drift is expensive for image data, we estimate it using Hutchinson's estimator [4]:

$$\nabla_{\mathbf{x}} \cdot \mathbf{v}(\mathbf{x}, t) \approx \boldsymbol{\epsilon}^T \frac{\partial \mathbf{v}(\mathbf{x}, t)}{\partial \mathbf{x}} \boldsymbol{\epsilon},$$

where $\mathbf{v}(\mathbf{x}, t)$ is the ODE drift and $\boldsymbol{\epsilon}$ is a random Gaussian vector. This avoids computing the full

Jacobian matrix.

**Code 3.1:** Implementation of the Hutchinson trace estimator used to approximate the divergence term in the Probability Flow ODE.

```
def divergence_hutchinson(drift_fn, x, t, noise_type="rademacher"):
    x = x.detach().requires_grad_(True)

    if noise_type == "rademacher":
        eps = torch.randint_like(x, low=0, high=2, dtype=torch.int64).float()
        eps = eps *2.0 -1.0
    elif noise_type == "gaussian":
        eps = torch.randn_like(x)
    else:
        raise ValueError("noise_type must be 'rademacher' or 'gaussian'")

    drift = drift_fn(x, t)
    inner = (drift *eps).reshape(x.shape[0], -1).sum(dim=1) # (B,)

    grad = torch.autograd.grad(
        outputs=inner.sum(),
        inputs=x,
        create_graph=False,
        retain_graph=False,
    )[0]

    div = (grad *eps).reshape(x.shape[0], -1).sum(dim=1)
    return drift.detach(), div.detach()
```

Finally, the log-likelihood estimate is converted into bits per dimension. We add a correction term of $+8$, because MNIST images are originally represented in the range $[0, 1]$, while the Probability Flow ODE maps them to a different continuous scale.

### 3.4.2. Fréchet Inception Distance

For FID, we used the `FrechetInceptionDistance` implementation from the TorchMetrics image metrics library instead of training a new Inception network ourselves. This was a practical decision, since FID depends on features extracted from a pretrained Inception model, and training such a model from scratch would be very expensive.

Before computing FID, we had to adapt the MNIST images to the input format expected by the Inception network. MNIST images are grayscale and have one channel, while the Inception network expects RGB images with three channels. For this reason, we rescaled the images and repeated the grayscale channel three times. We then computed FID by comparing the feature distribution of real MNIST images with the feature distribution of generated images.

**Code 3.2:** Rescaling MNIST images and adapting them from grayscale to RGB format.

```
def rescale(x):

    #If images are grayscale, repeat the channel 3 times
    if x.shape[1] == 1:
        x = x.repeat(1, 3, 1, 1)

    #Resize to the input size expected by Inception
    x = F.interpolate(
        x,
        size=(299, 299),
        mode="bilinear",
        align_corners=False,
    )

    # Clamp values to [0, 1]
    x = torch.clamp(x, 0.0, 1.0)

    #Convert to uint8 [0, 255]
    x = (x *255).to(torch.uint8)

    return x
```

### 3.4.3. Inception Score

For Inception Score, we used the `InceptionScore` implementation from the TorchMetrics image metrics library. As with FID, this implementation is based on a pretrained Inception network, which avoided the need to train a classifier ourselves.

The same preprocessing was applied before computing IS: the generated MNIST images were rescaled and converted from one grayscale channel to three channels. The Inception network then produced class probability distributions for the generated images. The score is higher when individual images are classified confidently and when the full set of generated samples is diverse.

## 3.5. Controllable Generation

In addition, we implemented two controllable generation experiments: class-conditional sampling and image imputation. These experiments were included to test whether the diffusion model could be guided by additional information during the reverse generation process.

### 3.5.1. Class-Conditional Sampling

For class-conditional sampling, we modified the original ScoreNet so that it also receives the digit label $y$ as an input. In the unconditional model, the network only depends on the noisy image $\mathbf{x}(t)$ and the time value $t$. In the conditional version, the model estimates a class-dependent score:

$$\mathbf{s}_\theta(\mathbf{x}(t), t, y).$$

To do this, we added a class embedding for the label $y$ and combined it with the time embedding before injecting it into the U-Net blocks. This allows the network to adapt its score prediction depending on the digit class that we want to generate.

**Code 3.3:** Modification of the ScoreNet architecture to include the digit label as an additional conditioning input.

```
class ConditionalScoreNet(nn.Module):
    def __init__(
        self,
        marginal_prob_std,
        n_classes=10,
        channels=[32, 64, 128, 256],
        embed_dim=256,
        in_channels=1,
    ):
        super().__init__()

        self.embed = nn.Sequential(
            GaussianRandomFourierFeatures(embed_dim=embed_dim),
            nn.Linear(embed_dim, embed_dim)
        )

        #New embedding
        self.class_embed = nn.Embedding(n_classes, embed_dim)
```

**Code 3.4:** Combination of the time embedding and class embedding in the forward pass of the conditional ScoreNet.

```
    def forward(self, x, t, y):
        #temporal embedding
        t_embed = self.act(self.embed(t))

        #target embedding
        y_embed = self.class_embed(y)

        #combine them
        embed = t_embed + y_embed
```

Since the architecture was changed, we retrained the model using MNIST images together with their corresponding labels. During sampling, we fix the desired class label and keep it constant throughout the reverse process. As a result, the generated image is guided not only toward the general MNIST distribution, but also toward the selected digit class.

### 3.5.2. Image Imputation

We used a binary mask to define which part of the image is observed and which part has to be generated. In our experiments, the mask keeps the left half of the image visible and hides the right half. Therefore, the model receives partial information about the digit and must complete the missing region in a plausible way.

To implement imputation, we adapted the reverse-time integrators so that the known pixels are reinserted after each update step. In Euler-Maruyama and Predictor-Corrector sampling, this correction is applied at every discrete time step after the sampler proposes a new image. For the Probability Flow ODE, we apply the same idea along the deterministic trajectory.

**Code 3.5:** Modification of the Euler-Maruyama to implement Imputation.

```
        # Noisy version of the observed image at the new time step
        sigma = sigma_t(t_next)
        x_obs_t = (
            mu_t(x_obs, t_next)
            + sigma.view(-1, 1, 1, 1) *z_obs[..., n + 1]
        )

        # Keep observed pixels from x_obs_t and generate only the missing ones
        x_t[..., n + 1] = (
            mask *x_obs_t
            + (1.0 -mask) *x_t[..., n + 1]
        )
```

**Code 3.6:** Modification of the Probability Flow ODE sampler to implement Imputation.

```
            z_obs = torch.randn_like(x_obs)
            sigma = sigma_t(t_next)
            x_obs_t = mu_t(x_obs, t_next) + sigma[:, None, None, None] *z_obs

            x = mask *x_obs_t + (1.0 -mask) *x
```

# 4

# Results

## 4.1. Unconditional Generation Results

In this section, we analyse the images generated by our trained model without class conditioning or imputation.

### 4.1.1. Variance Exploding Diffusion

Figures 4.1–4.3 show the generated images for the VE model using Euler-Maruyama, Predictor-Corrector, and Probability Flow ODE sampling. The VE model is able to generate digit-like structures correctly with both Euler-Maruyama and Predictor-Corrector sampling. Predictor-Corrector produces slightly cleaner samples because the corrector steps refine the image at each noise level. But as we mentioned, this improvement comes with a higher computational cost, since each corrector step requires additional evaluations of the ScoreNet. We also observed that the corrector step size must be chosen carefully: if it is too large, the Langevin correction can inject too much noise and degrade the generated image. In our experiments, the best value was $1{,}0 \times 10^{-4}$.

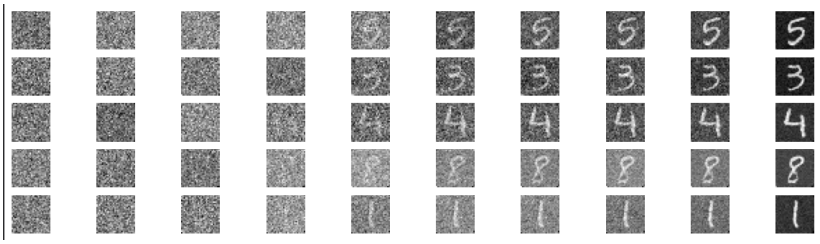


**Figure 4.1:** Samples generated with the VE and Euler-Maruyama sampling.

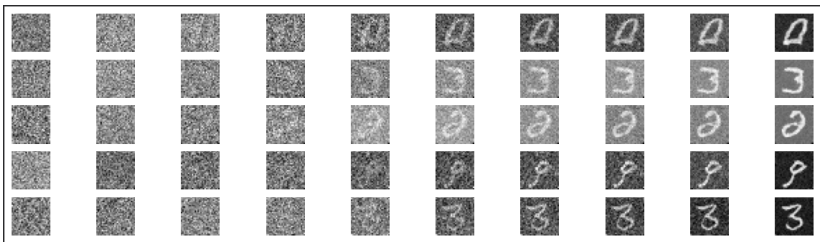


**Figure 4.2:** Samples generated with the VE and Predictor-Corrector sampling.

The Probability Flow ODE result is less successful. In our experiment, it only generates samples

that resemble digit one. A reasonable explanation is that, unlike the stochastic samplers, the ODE does not include an additional noise term that can help explore different regions of the distribution during generation. As a result, if the learned score field guides many trajectories toward a narrow region of the MNIST distribution, the ODE may produce less diverse samples.

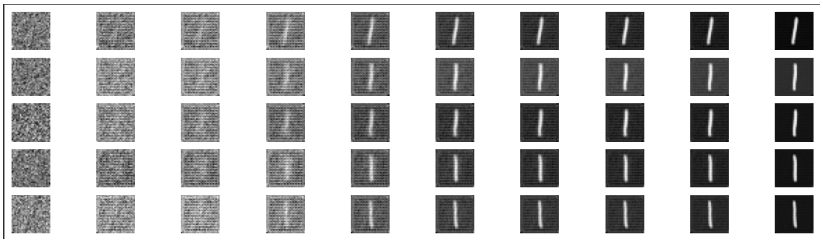


**Figure 4.3:** Samples generated with the VE model and Probability Flow ODE sampling.

### 4.1.2. Variance Preserving Diffusion with Linear Schedule

Figure 4.4 shows the result obtained with the VP-linear model using Predictor-Corrector sampling. The VP-linear model performs worse than the VE and VP-cosine models. As discussed previously in the theorical background, the linear schedule can remove useful image information, making the reverse generation problem harder, since the ScoreNet has to recover digit structure from noisier or less informative intermediate states. Because of this, the generated samples are less clear and less consistent. We only include the Predictor-Corrector result for this configuration, since it was the most visually informative among the VP-linear outputs.

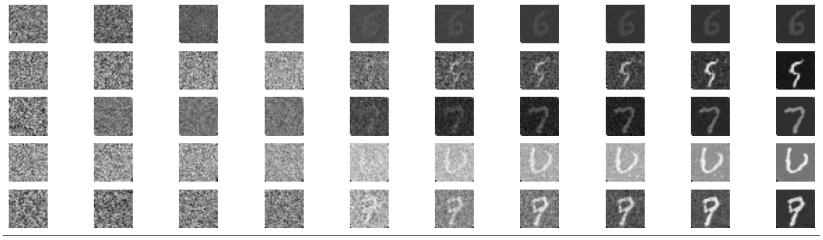


**Figure 4.4:** Samples generated with the VP-linear model and Predictor-Corrector sampling.

### 4.1.3. Variance Preserving Diffusion with Cosine Schedule

Figures 4.5, 4.6, and 4.7 show the results for the VP model with cosine schedule. As we said, the VP-cosine model gives better results than the VP-linear configuration, which suggests that the cosine schedule preserves useful image structure for longer during the forward diffusion process. Just like VE, the Euler-Maruyama sampler already produces recognisable digit shapes, but Predictor-Corrector slightly improves the visual quality.

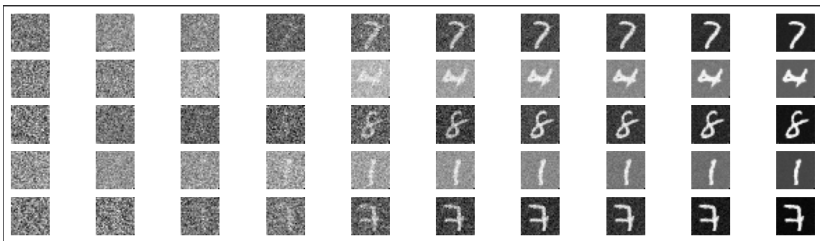


**Figure 4.5:** Samples generated with the VP-cosine model and Euler-Maruyama sampling.

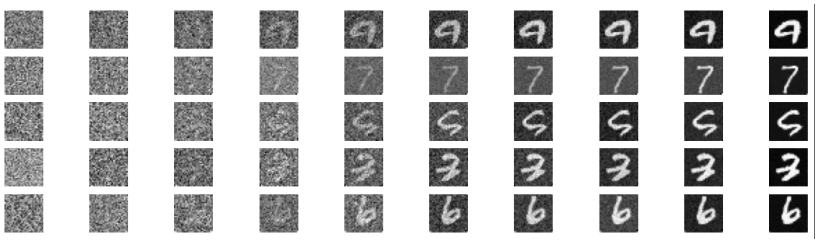


**Figure 4.6:** Samples generated with the VP-cosine model and Predictor-Corrector sampling.

The Probability Flow ODE also generates digit-like images for the VP-cosine model, but the results are generally weaker too. Small errors in the score approximation can accumulate along the trajectory, leading to less sharp or less diverse samples.

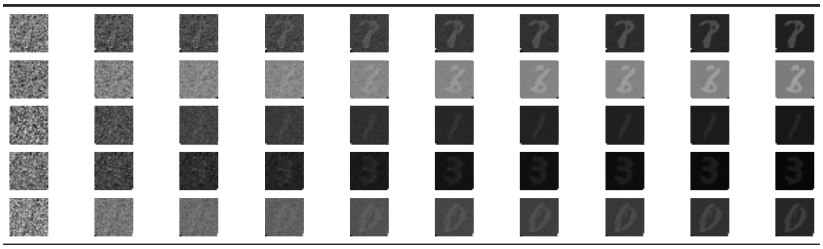


**Figure 4.7:** Samples generated with the VP-cosine model and Probability Flow ODE sampling.

Overall, the unconditional generation experiments show that the choice of noise schedule and sampler has a clear impact on image quality. VE and VP-cosine produce the best visual results, while VP-linear is less stable. Predictor-Corrector usually improves image quality compared with Euler-Maruyama, but it is slower and sensitive to the corrector step size. The Probability Flow ODE is useful from a theoretical and likelihood-estimation perspective, but in our experiments it generally performed worse as a visual sampler.

## 4.2. Quantitative Evaluation

In this section, we present the quantitative evaluation of the generated samples using BPD, FID and Inception Score. These metrics should be interpreted together with the visual results, because each

one evaluates a different aspect of the model.

### 4.2.1. Bits per Dimension

Let us recall, Bits per Dimension was computed using our Probability Flow ODE implementation. In this case, lower values indicate better likelihood. Table 4.1 shows the mean and standard deviation obtained for each diffusion configuration.

| Model | Mean BPD | Std. BPD |
|---|---|---|
| VE exponential | 4.4574 | 0.1977 |
| VP linear | 4.8735 | 0.6386 |
| VP cosine | 2.3062 | 0.3723 |

**Table 4.1:** Bits per Dimension results

The best result is obtained by the VP-cosine model, with a mean BPD of 2.3062. This suggests that, according to the likelihood-based evaluation, the cosine schedule provides a better probabilistic fit to the MNIST data. The VP-linear model obtains the highest BPD and also the largest standard deviation, which is consistent with the weaker visual results observed in the unconditional generation experiments.

### 4.2.2. Fréchet Inception Distance

For FID, we selected the best visual combination for each diffusion configuration: VE with Euler-Maruyama, VP-linear with Predictor-Corrector, and VP-cosine with Euler-Maruyama. Lower FID values indicate that the generated samples are closer to the real images in the feature space of the Inception network.

| Model | Sampler | FID |
|---|---|---|
| VE exponential | Euler-Maruyama | 170.0188 |
| VP linear | Predictor-Corrector | 176.1222 |
| VP cosine | Euler-Maruyama | 97.3110 |

**Table 4.2:** Fréchet Inception Distance results

The VP-cosine model obtains again the lowest FID, which agrees with the visual evaluation. Nevertheless, the absolute FID values are relatively high. This is expected because, as we said, FID is not especially well suited to MNIST. The metric uses an Inception network trained on natural RGB images, while our samples are grayscale handwritten digits that had to be adapted to three channels. We also observed that the number of evaluated images has a strong effect on the metric. When we evaluated only `n_images = 5` instead of a larger sample such as `n_images = 100`, the FID values became

much worse and less reliable. This happens because FID estimates means and covariances of feature distributions, so it needs a sufficiently large number of samples to produce stable results.

### 4.2.3. Inception Score

For Inception Score, we used the same selected combinations as in the FID evaluation. In this metric, higher values are better. The score increases when each generated image is assigned to a clear class with high confidence, and when the complete set of generated images covers different classes. In our case, this would mean that each generated digit should be easy to classify, while the whole sample should contain a variety of digits instead of repeating the same one.

| Model | Sampler | IS Mean | IS Std. |
|---|---|---|---|
| VE exponential | Euler-Maruyama | 1.9527 | 0.2510 |
| VP linear | Predictor-Corrector | 1.7664 | 0.0949 |
| VP cosine | Euler-Maruyama | 1.7571 | 0.2667 |

**Table 4.3:** Inception Score results for the selected model-sampler combinations

The Inception Score results are less intuitive than the FID and BPD results. Although VP-cosine performed best visually and obtained the best FID and BPD values, its IS is slightly lower than the VE result. This shows that IS also has limitations on this project, because the MNIST digits aren't natural RGB images.

In addition, IS is sensitive to the number of generated samples. When using only `n_images = 5`, the metric becomes unstable because the model cannot show enough class diversity in such a small sample.

## 4.3. Controllable Generation Results

In this section, we analyse the results obtained for the two controllable generation tasks: class-conditional sampling and image imputation. Unlike unconditional generation, these experiments introduce additional information into the generation process.

### 4.3.1. Class-Conditional Sampling

In the class-conditional, we used Euler-Maruyama sampling for all the examples, changing only the trained diffusion model and the target digit label. The objective was to check whether adding the label $y$ to the ScoreNet was enough to guide the reverse process toward a specific digit class.

For digit 3, we used the conditional VE model. As shown in Figure 4.8, the samples start from random noise and progressively become recognisable threes, suggesting that we were able to accomplish our objective.

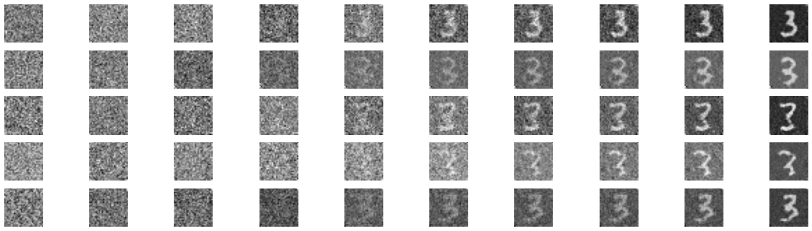


**Figure 4.8:** Class-conditional generation for digit 3 using the VE model with Euler-Maruyama sampling.

For digit 7, we also used Euler-Maruyama sampling, but with the conditional VP-linear model. As shown in Figure 4.9, the results are less stable than for digits 3, becuase this model performs slightly worse, but the target digit is still visible in some cases.

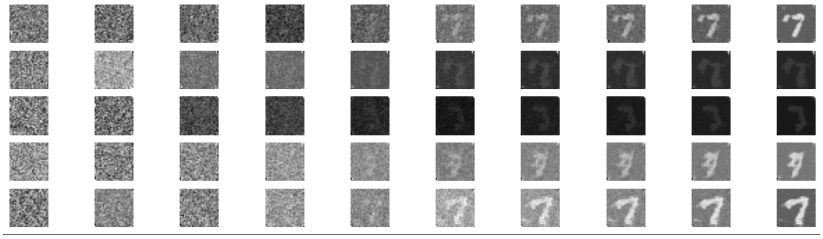


**Figure 4.9:** Class-conditional generation for digit 7 using the VP-linear model with Euler-Maruyama sampling.

Finally, for digit 4, we used the conditional VP-cosine model. Figure 4.10 also shows clear results.

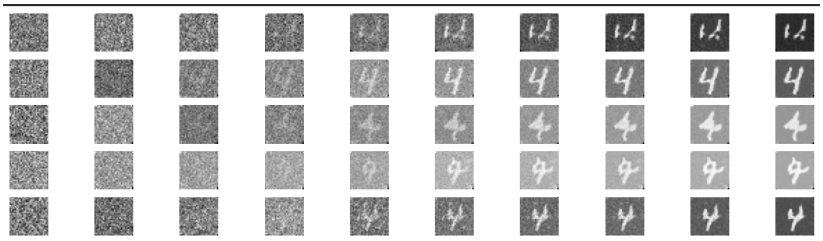


**Figure 4.10:** Class-conditional generation for digit 4 using the VP-cosine model with Euler-Maruyama sampling.

Overall, the results show that class conditioning works reasonably well. Since the label is included as an additional input to the ScoreNet, the estimated score is not only time-dependent, but also class-dependent.

### 4.3.2. Image Imputation

In the image imputation experiments, we first applied a mask to the original MNIST images. As explained, we use a binary mask that hides the right half of each digit and keeps the left half visible. The model then has to complete the missing right side using the visible left side as context. Figure 4.11 shows the original images and the corresponding masked inputs used in the experiment.

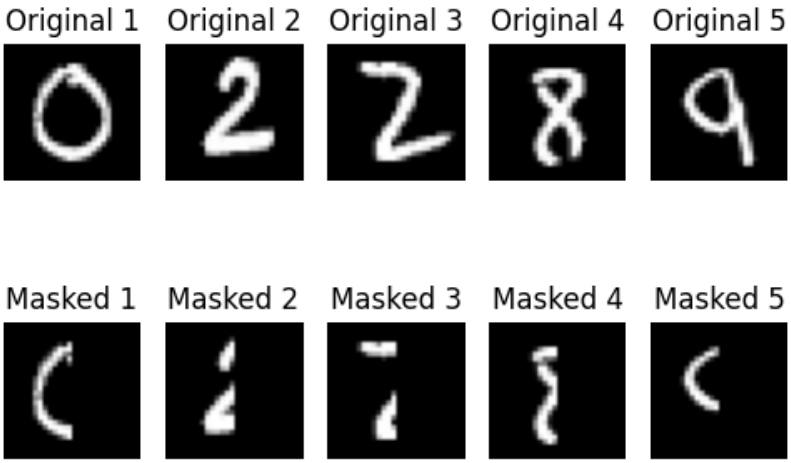



**Figure 4.11:** Original MNIST images and masked versions used for the imputation task.

To compare different behaviours, we used three model-sampler combinations. The first one was the VE model with Euler-Maruyama sampling. As shown in Figure 4.12, the model is able to complete most digits in an acceptable way, although not always matching the original digit exactly. A particularly interesting case appears in the third image: the original digit is a 2, but the completed image resembles a 3. This suggests that, with only the visible part of the digit, the model still found another completion that still looked reasonable, even though it did not match the original image exactly.



**Figure 4.12:** Image imputation using the VE model with Euler-Maruyama sampling.

The second combination was the VP-linear model with Predictor-Corrector sampling. Figure 4.13 shows that this method also produces coherent completions. The Predictor-Corrector sampler helps refine the generated region during the reverse process, and we can still see that the model also created a different digit for the third image.

The last combination was the VP-cosine model with Probability Flow ODE sampling. As shown in Figure 4.14, the generated completions are less sharp than the stochastic results. This is consistent



**Figure 4.13:** Image imputation using the VP-linear model with Predictor-Corrector sampling.

with the behaviour observed in unconditional generation, where the ODE sampler was useful but often produced weaker visual samples than Euler-Maruyama or Predictor-Corrector.



**Figure 4.14:** Image imputation using the VP-cosine model with Probability Flow ODE sampling.

Overall, the imputation results show that the trained models can use the visible pixels as context and generate plausible missing regions.

# 5

# Conclusions

In this project, we developed a generative AI system based on score-based diffusion models for MNIST image generation. The main objective was to understand how a neural network can be trained to estimate the score of noisy image distributions, and how this score can be used with stochastic differential equations to generate new images from noise.

Overall, VE and VP-cosine produced the most coherent digit-like samples. In contrast, VP-linear performed worse, probably because the linear schedule did not preserve image information as effectively during the forward process. This made the reverse generation task harder for the model. Among the samplers, Predictor-Corrector improved slightly visual quality compared with Euler-Maruyama, although it required more computation time and was sensitive to the corrector step size. The Probability Flow ODE was useful from a theoretical point of view and for likelihood-based evaluation, especially for computing BPD. However, as a visual sampler, it usually performed worse than the stochastic methods. This can be explained by the fact that the ODE follows a deterministic trajectory and is more sensitive to errors in the learned score field. In the VE case, this led to low diversity, with samples tending to resemble the digit one.

The quantitative evaluation confirmed some of the visual conclusions. The VP-cosine model achieved the best BPD and FID results, suggesting that it provided a better fit to the MNIST distribution. The Inception Score results were less consistent, which highlights the limitations of using Inception-based metrics on MNIST. Since the Inception network was trained on natural RGB images, and our data consists of grayscale handwritten digits, FID and IS should be interpreted only as complementary metrics.

We also extended the system with two controllable generation tasks. In class-conditional sampling, adding the digit label to the ScoreNet allowed us to guide the reverse process toward a selected class. The generated results showed that the model could produce digits consistent with the target label, although the quality varied depending on the class and diffusion configuration. In image imputation, adapting the reverse samplers with a binary mask allowed us to complete missing parts of MNIST images while preserving the observed pixels. Some completions did not exactly match the original digit, but they were still plausible given the visible region.

The main limitation of the project is that it was developed and evaluated on MNIST, which is much simpler than natural image datasets. As a result, the conclusions may not directly generalise to more complex images. In addition, training and evaluating diffusion models is computationally expensive, especially when using Predictor-Corrector sampling or the conditional modified ScoreNet.

As future work, we could extend the system to more complex datasets, such as CIFAR-10, train for more epochs, and increase the number of generated samples used in FID and IS. Finally, the imputation experiments could be improved by exploring more flexible masks.

# Bibliography


[1] Y. Song, "Generative modeling by estimating gradients of the data distribution." https://yang-song.net/blog/2021/score/, 2021. Accessed: 2026-05-03.

[2] J. M. Martínez, "Análisis de imágenes basado en deep learning: Fundamentos." Course slides, Tratamiento de Señales Visuales, Grado en Ciencia e Ingeniería de Datos, Universidad Autónoma de Madrid, 2026.

[3] A. Suárez, "Stochastic differential equations." Course slides, Departamento de Ciencias de la Computación, Universidad Autónoma de Madrid, n.d.

[4] M. F. Hutchinson, "A stochastic estimator of the trace of the influence matrix for laplacian smoothing splines," *Communications in Statistics - Simulation and Computation*, vol. 19, no. 2, pp. 433–450, 1990.


UÁM
Universidad Autónoma
de Madrid